

UNIVERSIDAD DEL VALLE DE GUATEMALA

Facultad de Ingeniería

Departamento de Ciencias de la Computación

Data Science

Erick Marroquín



Laboratorio 7 - Spark MLlib

Predicción del salario y segmentación de
asalariados con la ENEIC

Ricardo Arturo Godínez Sánchez - 23247

Vianka Vanessa Castro Ordoñez - 23201

Sebastian Bustamante - 22291

GUATEMALA, 26 de septiembre de 2026

1. Introducción

En este laboratorio trabajamos con las bases de **Personas** de la Encuesta Nacional de Empleo e Ingresos Continua (ENEIC) del Instituto Nacional de Estadística: los cuatro trimestres de 2025 y el primer trimestre de 2026. En total procesamos 253,519 registros distribuidos en cinco archivos de Excel. Nuestro objetivo fue doble: primero, entender la estructura salarial de los trabajadores asalariados del país; y segundo, construir modelos capaces de estimar el **salario mensual** a partir de seis características personales y laborales, evaluándolos al final con un trimestre que nunca usamos durante el desarrollo.

Todo el procesamiento lo hicimos con **Apache Spark 3.5.1** y su librería de aprendizaje automático, **MLlib**. Aunque el volumen de datos cabe en una computadora personal, el objetivo del laboratorio era aprender a pensar como en un entorno distribuido: expresar cada paso como transformaciones sobre DataFrames, calcular las métricas sobre los conjuntos completos y llevar a pandas únicamente tablas agregadas o muestras pequeñas para graficar.

El reporte sigue el orden en que avanzamos: preparación y calidad de datos, exploración, correlaciones, segmentación con KMeans, los dos pipelines de regresión, la evaluación final en 2026 y el análisis de errores. En cada etapa contamos qué hicimos, qué encontramos y cómo eso orientó la siguiente decisión.

1.1 Spark, Hadoop y nuestro entorno de trabajo

Antes de empezar dedicamos tiempo a entender la herramienta. **Hadoop** resolvió el problema de procesar datos que no caben en una sola máquina mediante un sistema de archivos distribuido (HDFS) y el modelo **MapReduce**: cada máquina procesa su parte de los datos (map), los resultados se agrupan por llave a través de la red (shuffle) y cada grupo se combina (reduce). Su debilidad es que escribe a disco entre cada paso. **Spark** generaliza esa idea: mantiene los resultados intermedios en memoria, construye un plan completo de operaciones (un DAG) y lo optimiza antes de ejecutarlo. Esto último se nota en el código, porque las *transformaciones* (`filter`, `withColumn`, `groupBy`) no se ejecutan hasta que aparece una *acción* (`count`, `toPandas`, `write`).

En nuestro caso no hubo un clúster Hadoop: Spark corrió en modo `local[2]` dentro de un contenedor Docker con Python 3.11, Java 17 y PySpark 3.5.1. Nos dimos cuenta de que el contenedor era indispensable porque Spark se ejecuta sobre la máquina virtual de Java y las versiones de pandas y numpy deben ser compatibles con PySpark 3.5; con Docker todos los integrantes trabajamos exactamente con el mismo entorno. Además, el código que escribimos es el mismo que correría en un clúster real: sólo cambiaría el parámetro `master`.

2. Carga, armonización y calidad de datos

Spark no tiene un lector nativo de Excel, así que leímos cada archivo con pandas y openpyxl, **uno a la vez**, todo como texto. Inmediatamente lo convertimos a un DataFrame de Spark con un esquema explícito, agregamos la procedencia del registro, homologamos los tipos y lo guardamos en Parquet, un formato columnar y comprimido que Spark lee de forma muy eficiente. A partir de ese punto todo el trabajo se hizo sobre Parquet.

Tabla 1. Dimensiones de los archivos originales.

Archivo	Filas	Columnas
2025T1	51,588	270
2025T2	51,167	270
2025T3	51,583	270
2025T4	49,338	302
2026T1	49,843	270

La primera sorpresa fue que el archivo del cuarto trimestre de 2025 tiene **302 columnas**, mientras los demás tienen 270. Si hubiéramos apilado los archivos por posición, una columna de edad podría haber terminado mezclada con otra variable sin ningún aviso. Por eso seleccionamos las columnas requeridas y unimos los cuatro trimestres de 2025 con `unionByName`, que empareja por nombre y no por posición.

La segunda sorpresa estuvo en la columna `TRIMESTRE`. Su valor no coincide con el trimestre calendario: el archivo del primer trimestre dice 2, el del segundo dice 3, y así sucesivamente. Restar 1 parecía la solución, pero en el archivo de 2025T2 encontramos **175 registros con el código 2**. Esa operación los habría clasificado como del primer trimestre. Decidimos conservar la columna original sin modificarla y crear `periodo_archivo`, `anio_archivo` y `trimestre_calendario` a partir del archivo de procedencia, junto con `archivo_origen` para asegurar la trazabilidad.

Tabla 2. Valores de `TRIMESTRE` observados en cada archivo.

Archivo	Valor original de TRIMESTRE	Registros
2025T1	2	51,588
2025T2	2	175
2025T2	3	50,992
2025T3	4	51,583
2025T4	5	49,338
2026T1	6	49,843

2.1 Faltantes, códigos y duplicados

Antes de filtrar contamos los faltantes de cada variable seleccionada. El salario (P05D01) falta en el **73.97%** de los registros de 2025, y las variables laborales (categoría ocupacional, horas y antigüedad) en cerca del 57%. Al principio nos pareció alarmante, pero al revisar el cuestionario entendimos que la mayoría no son errores: son preguntas que **no corresponden** a niños, estudiantes, personas desempleadas o trabajadores por cuenta propia. Aprendimos a distinguir entre un dato ausente porque la pregunta no aplica y una respuesta que debía registrarse y no se registró; la diferencia se establece con el flujo del cuestionario, no sólo observando el nulo.

Los códigos llegaban a veces como número y a veces como texto (1, 1.0, "1"), así que los normalizamos antes de unir y los validamos contra el diccionario de cada período. Los códigos ausentes o no reconocidos quedaron como **DESCONOCIDO**, cuidando que el código educativo 0 se conservara como "ninguno" y no como faltante. Finalmente verificamos la unicidad de la combinación período, hogar y persona: encontramos **0 claves repetidas** y 0 claves incompletas, por lo que no fue necesario investigar conflictos ni usar `dropDuplicates`.

2.2 Población analítica y filtros

Limitamos el análisis a personas de 15 años o más, ocupadas, asalariadas (categorías 1 a 4 de P05C16) y con salario positivo, y aplicamos las reglas de calidad para edad, antigüedad (en años más meses/12) y horas semanales. Para contabilizar cada exclusión en el mismo orden no filtramos ocho veces: creamos una columna con el número del primer criterio que falla cada registro y resolvimos todo el conteo con un solo `groupBy`.

Tabla 3. Registros excluidos en cada paso de filtrado (2025, orden fijo).

Paso	Criterio	Excluidos en 2025
1	Edad finita y al menos 15	62,886
2	Ocupado = 1	52,568
3	Asalariado: categorías 1–4	35,197
4	Salario finito y positivo	0
5	Años de antigüedad finitos y no negativos	0
6	Meses enteros de 0 a 11	0
7	Antigüedad finita y no mayor a edad	0
8	Horas finitas, mayores a 0 y hasta 168	0

Tabla 4. Registros por archivo antes y después de los filtros.

Archivo	Antes	Después	Conservado
2025T1	51,588	13,419	26.0%
2025T2	51,167	13,492	26.4%
2025T3	51,583	13,450	26.1%
2025T4	49,338	12,664	25.7%
2026T1	49,843	13,258	26.6%

De 203,676 registros de 2025 conservamos **53,025** (26.03%); para 2026T1 quedaron 13,258. Nos llamó la atención que los filtros 4 a 8 no excluyeran ningún registro: una vez delimitada la población asalariada, las variables numéricas ya estaban completas y dentro de rango. Mantuvimos esos filtros como control. También entendimos que estas exclusiones **definen la población de estudio** y no son errores de calidad, y que esta base no representa a todos los trabajadores del país: es una muestra, sólo incluye asalariados con salario positivo registrado y la analizamos sin ponderar por el factor de expansión. Por otro lado, una persona observada en dos trimestres no es un duplicado: son dos observaciones de una trayectoria en la que su salario o empleo pudo cambiar.

Guardamos por separado los conjuntos preparados de 2025 y 2026 en Parquet. Desde ese momento, 2026 quedó reservado y no lo volvimos a abrir hasta la evaluación final.

3. Estadística descriptiva y exploración

Con la población analítica de 2025 calculamos los estadísticos descriptivos directamente en Spark, con percentiles exactos sobre los 53,025 registros.

Tabla 5. Estadística descriptiva de la población analítica de 2025 ($n = 53,025$).

Variable	Media	Mediana	Desv. est.	Mín.	P25	P75	P95	Máx.
Salario (Q)	3,421.7	3,000.0	2,902.1	1.0	1,800.0	4,000.0	8,000.0	99,000.0
Edad	35.2	33.0	13.5	15.0	24.0	44.0	61.0	89.0
Antigüedad (años)	5.6	2.0	7.8	0.0	0.5	7.0	22.0	70.0
Horas/semana	46.3	45.0	18.6	1.0	40.0	55.0	80.0	126.0

Lo primero que observamos fue la forma del salario. La media es Q3,422 y la mediana Q3,000: una diferencia de Q422. La mitad central de los trabajadores gana entre Q1,800 y Q4,000, pero el máximo llega a Q99,000, más de doce veces el percentil 95. La asimetría es **6.00**: la distribución está fuertemente sesgada a la derecha, con una cola larga de salarios altos que jala la media hacia arriba. Por eso la mediana describe mejor al trabajador típico. Siguiendo las instrucciones, no eliminamos esos extremos; más adelante veremos que explican buena parte de los errores de los modelos.

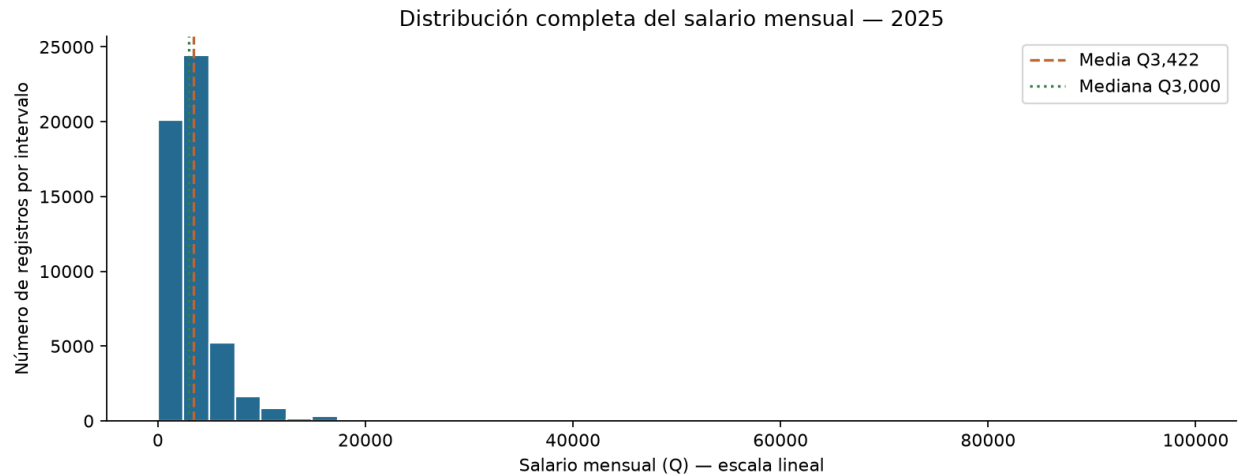


Figura 1. Distribución completa del salario mensual en escala lineal (2025).

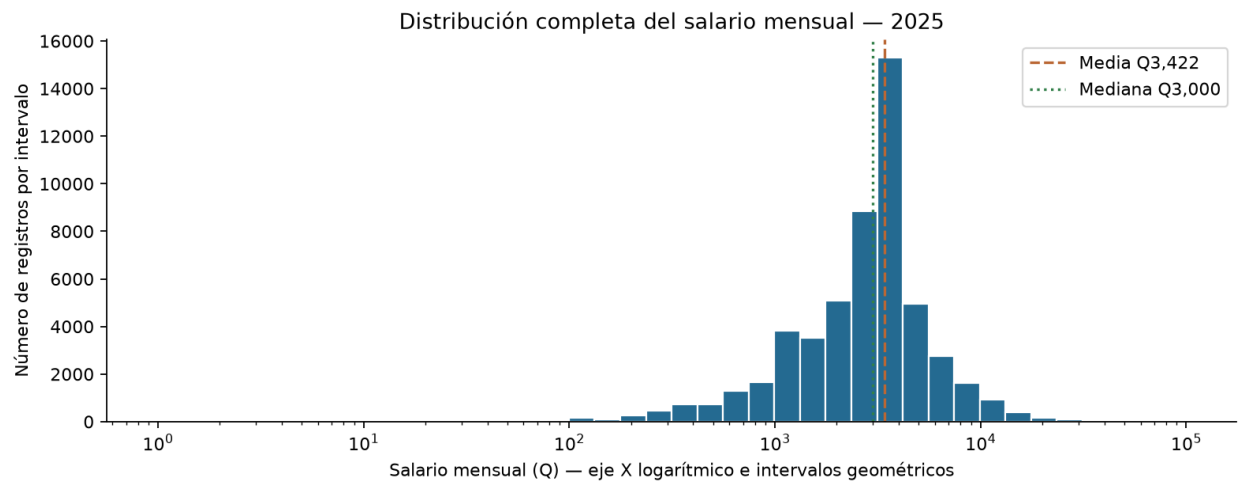


Figura 2. Distribución del salario mensual con **eje X logarítmico**. La escala sólo cambia la visualización; el objetivo de los modelos se mantuvo en quetzales.

Para graficar sin llevar la base completa a pandas, calculamos en Spark a qué intervalo pertenece cada salario y agrupamos: a pandas sólo viajaron 40 conteos. Esta técnica de agregar en Spark y dibujar en pandas la usamos en todo el laboratorio.

3.1 Distribución por categorías y salario mediano

Tabla 6. Distribución de los registros de 2025 y salario mediano por grupo.

Variable	Grupo	Registros	%	Salario mediano
Categoría	Empleado de gobierno	6,575	12.4%	Q5,000
Categoría	Empleado de empresa privada	28,702	54.1%	Q3,568
Categoría	Empleado jornalero o peón	13,842	26.1%	Q1,800
Categoría	Del servicio doméstico	3,906	7.4%	Q1,000
Educación	Ninguno	3,995	7.5%	Q1,500
Educación	Preprimaria	459	0.9%	Q2,160
Educación	Primaria	15,822	29.8%	Q2,200
Educación	Básico	8,249	15.6%	Q2,800
Educación	Diversificado	16,851	31.8%	Q3,600
Educación	Superior	6,818	12.9%	Q5,000
Educación	Maestría	771	1.5%	Q10,000
Educación	Doctorado	60	0.1%	Q12,000
Dominio	Urbano metropolitano	22,072	41.6%	Q3,700
Dominio	Resto urbano	20,146	38.0%	Q2,880
Dominio	Rural nacional	10,807	20.4%	Q2,000

La mayoría de los asalariados trabaja en empresas privadas (54.1%) y el nivel educativo más frecuente es diversificado. El hallazgo más claro fue la **escalera salarial por educación**: la mediana pasa de Q1,500 para quienes no tienen estudios a Q12,000 con doctorado, aunque este último grupo sólo tiene 60 registros. Por categoría, los empleados de gobierno tienen una mediana de Q5,000 frente a Q1,000 en el servicio doméstico, y el área metropolitana casi duplica al área rural. Estas diferencias son descriptivas: todavía no están ajustadas por edad, antigüedad u horas.

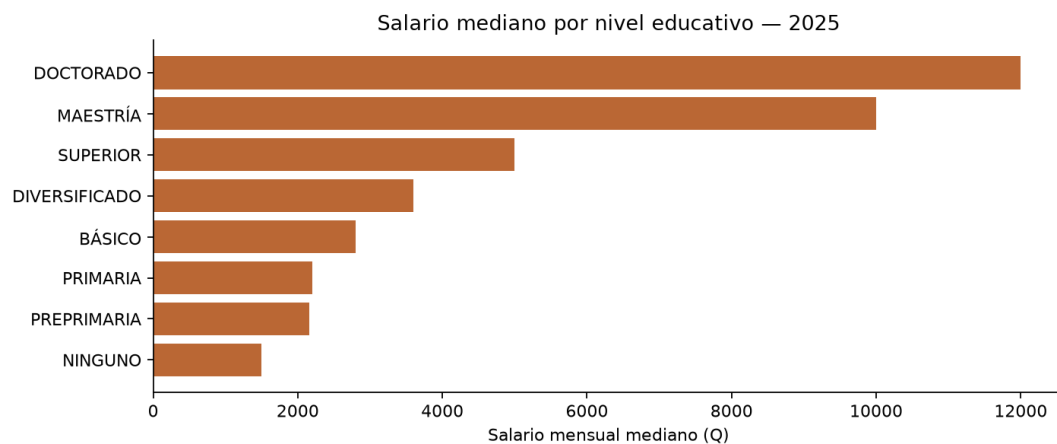


Figura 3. Salario mediano por nivel educativo (2025).

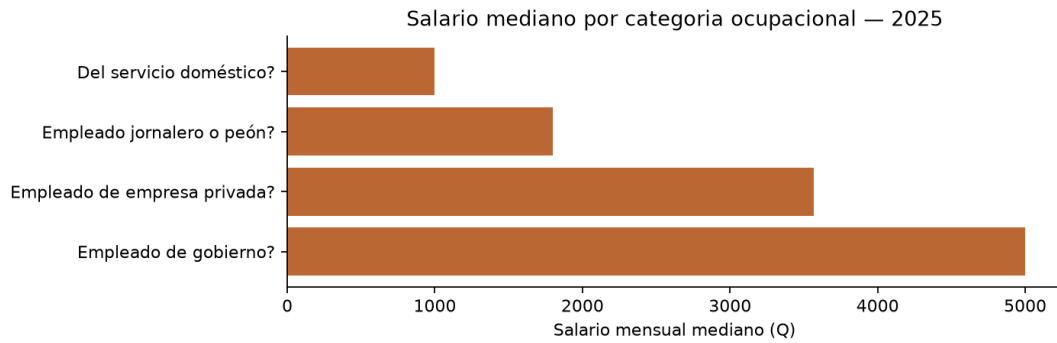


Figura 4. Salario mediano por categoría ocupacional (2025).

3.2 Evolución entre trimestres

Entre el primer y el cuarto trimestre de 2025 la muestra analítica pasó de 13,419 a 12,664 registros y la mediana salarial de Q3,000 a Q3,200. Nos cuidamos de interpretar este cambio como un aumento de sueldo: cada trimestre observa a personas distintas por la rotación de la muestra y los salarios son nominales, sin ajustar por inflación.

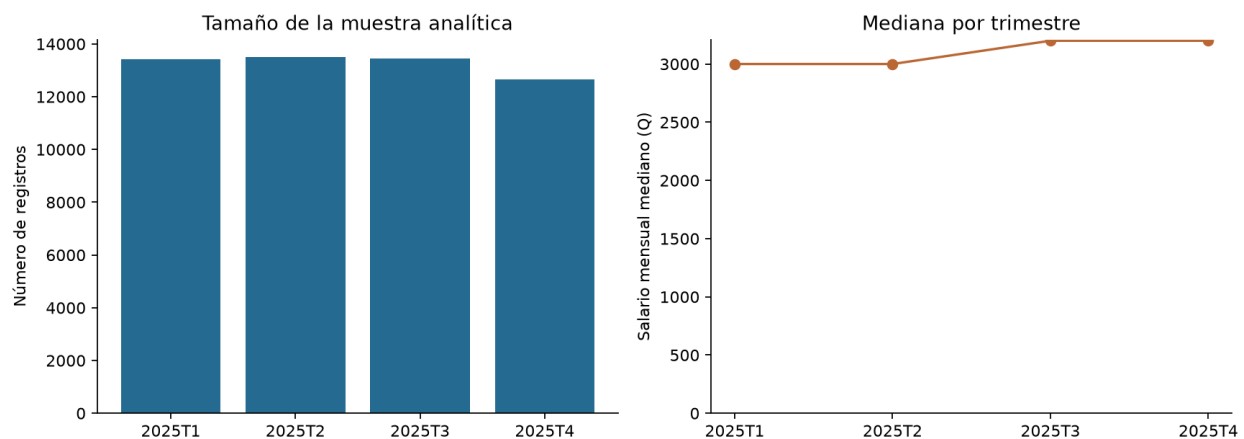


Figura 5. Tamaño de la muestra analítica y salario mediano por trimestre de 2025.

4. Relaciones entre variables numéricas

Para medir la asociación lineal usamos `VectorAssembler`, que junta las cuatro variables en una sola columna de tipo vector, y `Correlation.corr` de `pyspark.ml.stat`, calculando la correlación de Pearson sobre todos los registros elegibles de 2025.

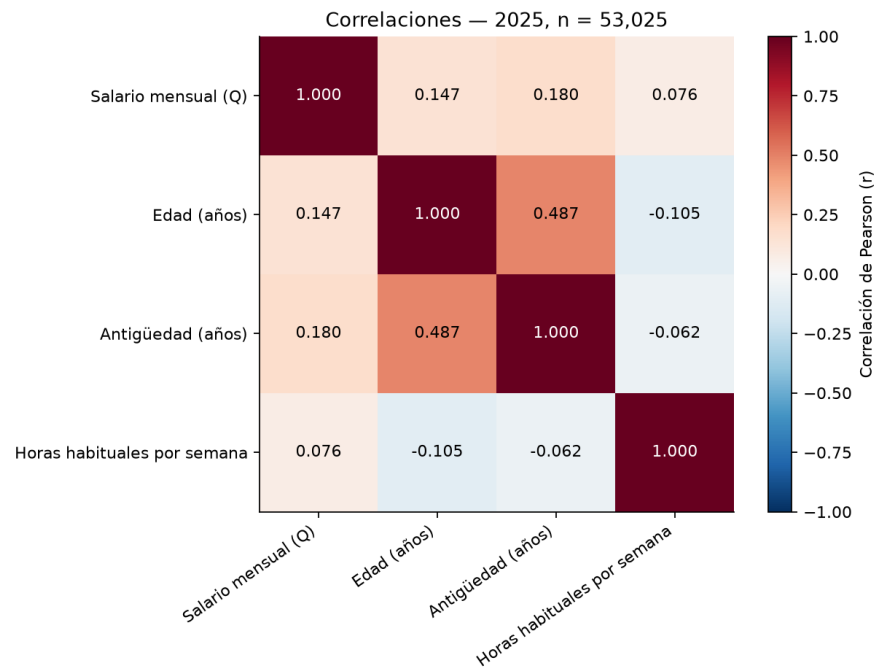


Figura 6. Matriz de correlaciones de Pearson (2025).

El resultado nos hizo ajustar las expectativas. La variable con mayor asociación lineal con el salario es la **antigüedad** ($r = 0.180$), seguida de la edad ($r = 0.147$) y las horas semanales ($r = 0.076$). Todas son asociaciones **débiles**. En cambio, edad y antigüedad sí están relacionadas ($r = 0.487$), algo lógico porque para acumular años en un puesto hay que tener edad. Gracias a esto anticipamos que las variables numéricas por sí solas explicarían poco del salario y que las categóricas, en especial educación y categoría ocupacional, tendrían que cargar con buena parte del poder predictivo.

5. Segmentación de perfiles con KMeans

Quisimos saber si existen perfiles laborales distintos entre los asalariados. KMeans agrupa por distancia euclidiana, así que usamos variables numéricas continuas: **edad**, **antigüedad** y **horas semanales**. Dejamos fuera las categóricas porque la distancia entre códigos one-hot no tiene un significado real; las usamos después para describir los grupos. Todas las variables se estandarizaron con `StandardScaler` dentro de un `Pipeline`, para que ninguna dominara por su escala.

5.1 ¿Vale la pena incluir el salario?

Antes de decidir revisamos la forma de cada variable. Estandarizar iguala escalas pero no elimina colas: el salario máximo queda a **32.9 desviaciones estándar** de la media, mientras que edad, antigüedad y horas no pasan de 8.2. Por eso comparamos tres escenarios con $K = 2, 3, 4$ y 5 : sin salario, con salario en quetzales y con el logaritmo del salario.

Tabla 7. Silhouette por escenario y número de clusters.

K	A: sin salario	B: salario en Q	C: log10(salario)
2	0.554	0.515	0.471
3	0.443	0.337	0.475
4	0.476	0.370	0.372
5	0.484	0.384	0.399
Promedio	0.489	0.402	0.430

Sin salario la silhouette fue mayor en los cuatro valores de K frente al salario en quetzales, y en tres de cuatro frente al logaritmo. Además, con salario en quetzales y $K = 5$ apareció un cluster con sólo 4.0% de los registros, formado por salarios extremos. Decidimos **no incluir el salario para formar los grupos**. Esta decisión tuvo una ventaja adicional: los clusters representan perfiles laborales y después pudimos comprobar si esos perfiles ganan distinto sin haberlo forzado.

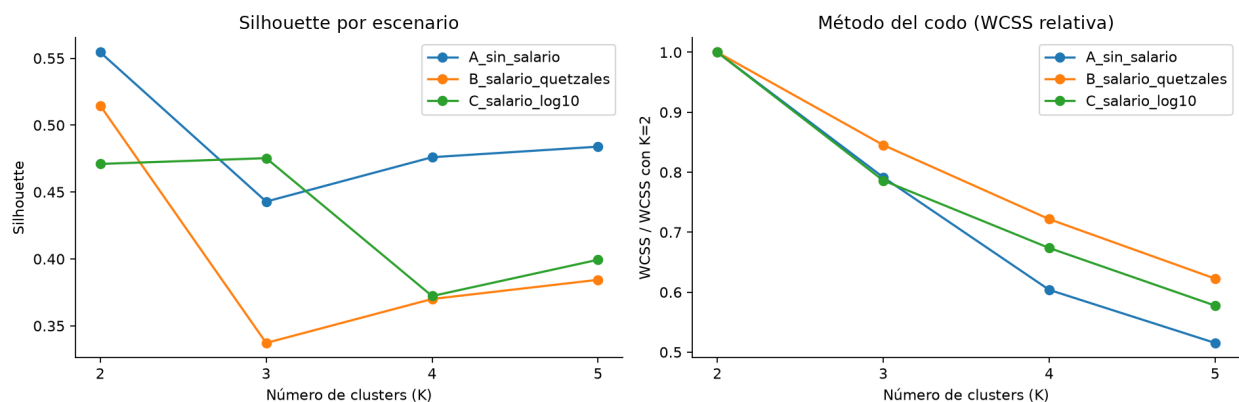


Figura 7. Silhouette y método del codo (WCSS relativa a $K = 2$) por escenario.

5.2 Elección de K

Como grupo definimos el criterio: K de al menos 3, porque buscábamos perfiles y no sólo partir la muestra en dos; ningún cluster con menos del 5% de los registros; y, entre los que cumplen, la mayor silhouette. K = 2 tuvo la silhouette más alta (0.554), pero sólo separaba a los jóvenes con poca antigüedad de los mayores con más antigüedad, la misma relación que ya habíamos visto en las correlaciones. Con el criterio elegimos **K = 5** (silhouette 0.484), que además coincide con la zona del codo.

5.3 Los perfiles encontrados

Tabla 8. Perfil de cada cluster (medianas de edad, antigüedad en años y horas semanales).

Cluster	%	Edad	Antig.	Horas	Salario mediano	% gobierno	% rural
Veteranos estables	12.5%	49	20.0	40	Q3,800	31.5%	16.5%
Jornada extensa	11.7%	30	2.0	76	Q3,000	9.5%	23.1%
Jornada parcial	12.3%	30	1.2	21	Q1,200	14.7%	28.0%
Adultos con poca antigüedad	20.8%	48	3.0	44	Q3,200	12.9%	17.0%
Jóvenes en inicio laboral	42.8%	26	1.5	48	Q3,200	6.7%	20.2%

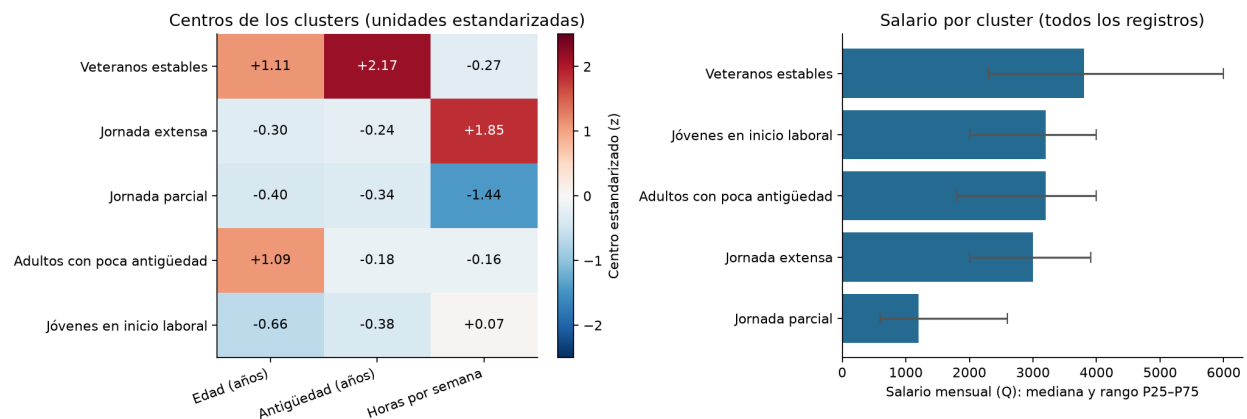


Figura 8. Centros estandarizados de cada cluster y salario por cluster (mediana y rango P25–P75).

Asignamos los nombres con una regla explícita sobre los centros estandarizados: un valor mayor a +0.5 o menor a −0.5 desviaciones marca el rasgo distintivo del grupo. Así surgieron cinco perfiles:

- **Jóvenes en inicio laboral** (42.8%): el grupo más grande; alrededor de 26 años, poca antigüedad y jornada completa, mayoritariamente en empresa privada.
- **Adultos con poca antigüedad**: personas cercanas a los 48 años que llevan pocos años en su trabajo actual.
- **Veteranos estables**: unos 20 años en el mismo puesto; es el grupo con más empleados de gobierno y con más educación superior.
- **Jornada extensa**: trabajan alrededor de 76 horas por semana, casi el doble de lo habitual.

- **Jornada parcial:** alrededor de 21 horas semanales, con alta presencia de jornaleros, servicio doméstico y área rural.

Lo más interesante fue que, aunque el salario no participó en la segmentación, las medianas salariales van de **Q1,200** en *Jornada parcial* a **Q3,800** en *Veteranos estables*, 3.2 veces más. También nos llamó la atención que el grupo de jornada extensa no gana más que los jóvenes con jornada normal. Una silhouette cercana a 0.48 indica grupos distinguibles pero con traslape: los perfiles laborales forman un continuo y los clusters son una simplificación útil. El cluster no se usó como predictor en los modelos supervisados.

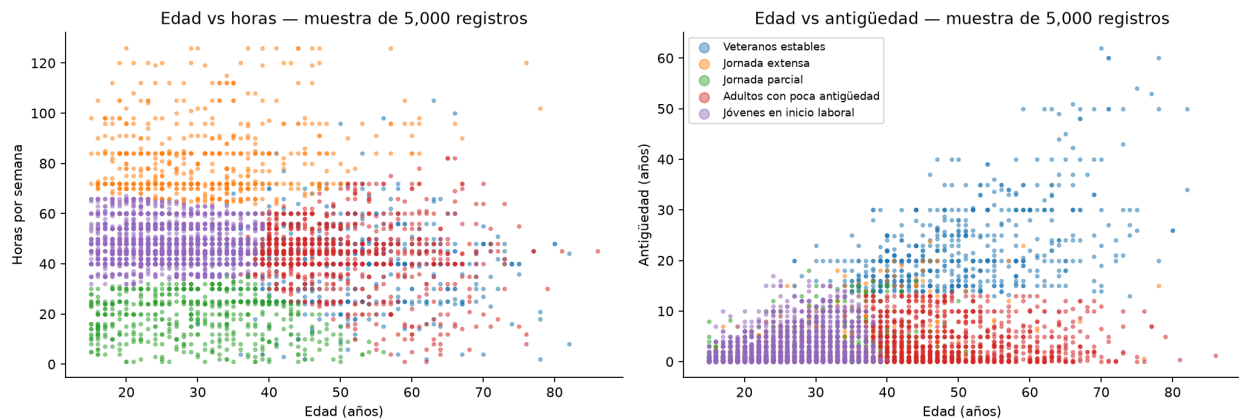


Figura 9. Clusters sobre una muestra de 5,000 registros (sólo para visualizar).

6. Modelos de regresión con pipelines

La tarea supervisada fue estimar `salario_mensual` a partir de exactamente seis predictores: edad, antigüedad, horas semanales, nivel educativo, categoría ocupacional y dominio. Excluimos identificadores, el factor de expansión, otros ingresos y la etiqueta de cluster.

6.1 Diseño experimental

Dividimos por tiempo y no al azar: entrenamiento con 2025T1–T3 (40,361 registros), validación con 2025T4 (12,664) y 2026T1 reservado para la prueba final. Queríamos comprobar si lo aprendido en unos trimestres se sostiene en uno posterior, que es la situación real de uso. Como referencia construimos un **baseline** que predice siempre la media salarial del entrenamiento: su RMSE de validación fue Q2,890 con $R^2 \approx 0$. Cualquier modelo útil debía superar ese número.

Ambos pipelines comparten el preprocesamiento: `StringIndexer` y `OneHotEncoder` para las tres categóricas, y `VectorAssembler` para juntar todo en una columna `features`. Poner el preprocesamiento dentro del pipeline garantiza que los índices de categorías se aprendan sólo con el entrenamiento, evitando fuga de información. En Spark, al hacer `pipeline.fit()` cada etapa se convierte en un modelo ya entrenado.

```
StringIndexer -> OneHotEncoder -> VectorAssembler -> LinearRegression |
RandomForestRegressor
```

6.2 Regresión lineal

Usamos `standardization=True` para estandarizar internamente los predictores y probamos siete configuraciones de regularización: sin penalización, Ridge y Elastic Net.

Tabla 9. Configuraciones de regresión lineal evaluadas en 2025T4 (ordenadas por RMSE).

Configuración	regParam	elasticNet	MAE val.	RMSE val.	R ² val.
sin_regularizacion	0.0	0.0	Q1,210	Q2,186.42	0.4257
ridge_0.10	0.1	0.0	Q1,210	Q2,186.42	0.4257
ridge_1.00	1.0	0.0	Q1,210	Q2,186.43	0.4257
elastic_net_1.00	1.0	0.5	Q1,210	Q2,186.47	0.4256
ridge_10.00	10.0	0.0	Q1,210	Q2,186.55	0.4256
elastic_net_10.00	10.0	0.5	Q1,208	Q2,186.89	0.4254
ridge_50.00	50.0	0.0	Q1,208	Q2,187.15	0.4253

La mejor fue **sin_regularizacion**, con RMSE de Q2,186, MAE de Q1,210 y R² de 0.426: un 24.3% menos de error que el baseline. Nos dimos cuenta de que la regularización prácticamente no cambia nada (menos de un quetzal entre configuraciones). La razón es que con más de 40 mil registros y alrededor de 18 variables el modelo no tiene cómo sobreajustar; de hecho, su RMSE de validación fue 2.6% menor que el de entrenamiento.

6.3 Random Forest

El bosque aleatorio no necesita estandarización porque sus árboles dividen por umbrales. Probamos cinco combinaciones de número de árboles y profundidad máxima, todas con la semilla 2026.

Tabla 10. Configuraciones de Random Forest evaluadas en 2025T4 (ordenadas por RMSE).

Configuración	Árboles	Prof.	RMSE entr.	RMSE val.	R ² val.	Brecha
rf_100_arboles_d14	100	14	Q1,878	Q1,959	0.539	+4.3%
rf_200_arboles_d12	200	12	Q1,916	Q1,973	0.532	+3.0%
rf_100_arboles_d12	100	12	Q1,917	Q1,979	0.530	+3.2%
rf_100_arboles_d10	100	10	Q1,977	Q2,006	0.517	+1.4%
rf_50_arboles_d8	50	8	Q2,058	Q2,048	0.496	-0.5%

A mayor profundidad el error bajó, pero la brecha entre entrenamiento y validación creció hasta 4.3%: el bosque empieza a memorizar, aunque todavía en un nivel aceptable. También vimos que la profundidad importa más que la cantidad de árboles: duplicar de 100 a 200 árboles mejoró apenas unos quetzales y tardó el doble. La mejor configuración fue **rf_100_arboles_d14**, con RMSE de Q1,959 y R² de 0.539. En la interfaz de Spark observamos cómo el tamaño de los árboles que se envían a los ejecutores crecía con la profundidad, lo que explica por qué los bosques profundos tardan más.

Tabla 11. Comparación en validación (2025T4).

Modelo	Configuración	MAE	RMSE	R ²	Mejora vs baseline
Baseline	Media del entrenamiento	Q1,673	Q2,890	-0.003	—
Regresión lineal	sin_regularizacion	Q1,210	Q2,186	0.426	24.3%
Random Forest	rf_100_arboles_d14	Q1,058	Q1,959	0.539	32.2%

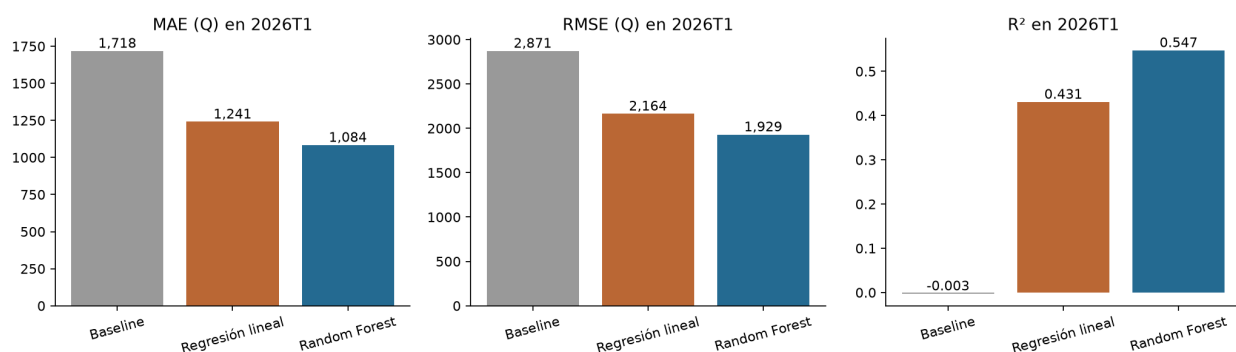
Random Forest obtuvo un RMSE cerca de 10% menor que la regresión lineal. Lo atribuimos a que la regresión lineal supone efectos aditivos (tener educación superior suma lo mismo en cualquier contexto), mientras que el bosque representa interacciones y relaciones no lineales, por ejemplo que la educación valga más en el área metropolitana o que la antigüedad pese distinto en el gobierno que en el servicio doméstico.

7. Entrenamiento final y evaluación en 2026

Con una configuración elegida por algoritmo, reentrenamos ambos pipelines con **todo 2025** (53,025 registros) y los evaluamos una sola vez en **2026T1** (13,258 registros), preparado con las mismas reglas del inicio. Los hiperparámetros se leyeron del archivo de selección, sin volver a elegirlos con 2026. Para garantizar que ambos modelos y el baseline se evaluarán sobre exactamente los mismos registros, unimos sus predicciones en una sola tabla por la clave hogar-persona y verificamos que no se perdiera ninguna fila. En 2026 no apareció ninguna categoría nueva.

Tabla 12. Métricas en la prueba final 2026T1, comparadas con la validación.

Modelo	MAE	RMSE	R ²	RMSE validación 2025T4	Cambio de RMSE
Baseline	Q1,718	Q2,871	-0.003	Q2,890	-0.6%
Regresión lineal	Q1,241	Q2,164	0.431	Q2,186	-1.0%
Random Forest	Q1,084	Q1,929	0.547	Q1,959	-1.5%

Figura 10. MAE, RMSE y R² de los tres modelos en 2026T1.

Random Forest volvió a ganar: MAE de Q1,084, RMSE de Q1,929 y R² de 0.547, un 32.8% menos de error que el baseline, frente a 24.6% de la regresión lineal. Lo que más nos tranquilizó fue que el

desempeño se sostuvo en un período nuevo: el RMSE incluso bajó 1.5% respecto a la validación, en parte porque los modelos finales se entrenaron con 25% más datos. Esto confirma que la selección hecha en 2025T4 fue adecuada. Un detalle que notamos: la regresión lineal produjo 29 predicciones negativas, algo imposible para un salario, mientras que un árbol nunca predice fuera del rango observado.

8. Visualización y análisis de errores

Definimos el residuo una sola vez como **salario real – salario predicho**: un residuo positivo indica que el modelo **subestimó** y uno negativo que **sobreestimó**. Para comparar visualmente ambos modelos usamos la misma muestra aleatoria de 5,000 registros; todas las métricas por grupo las calculamos con los registros completos de prueba.

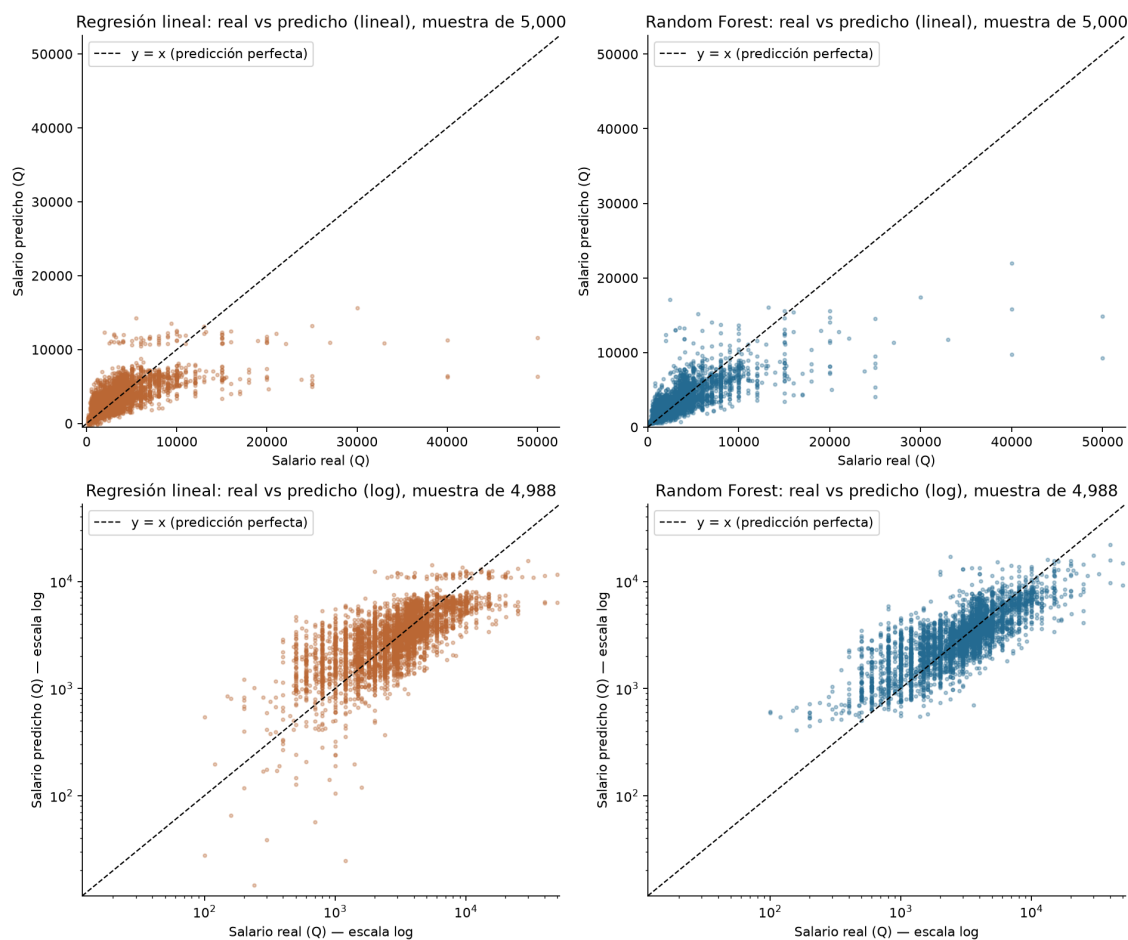


Figura 11. Salario real frente a predicho con la línea $y = x$. Arriba en escala lineal; abajo con ejes logarítmicos. Muestra de 5,000 registros de 2026T1.

La primera gráfica fue reveladora: los puntos no siguen la línea $y = x$ sino que forman una nube **aplanada**. Personas que ganan Q40,000 reciben predicciones cercanas a Q10,000, y quienes ganan menos de Q1,000 reciben predicciones de más de Q1,500. En escala logarítmica se ve que Random Forest sigue mejor la diagonal que la regresión lineal, aunque ambos comprimen los extremos.

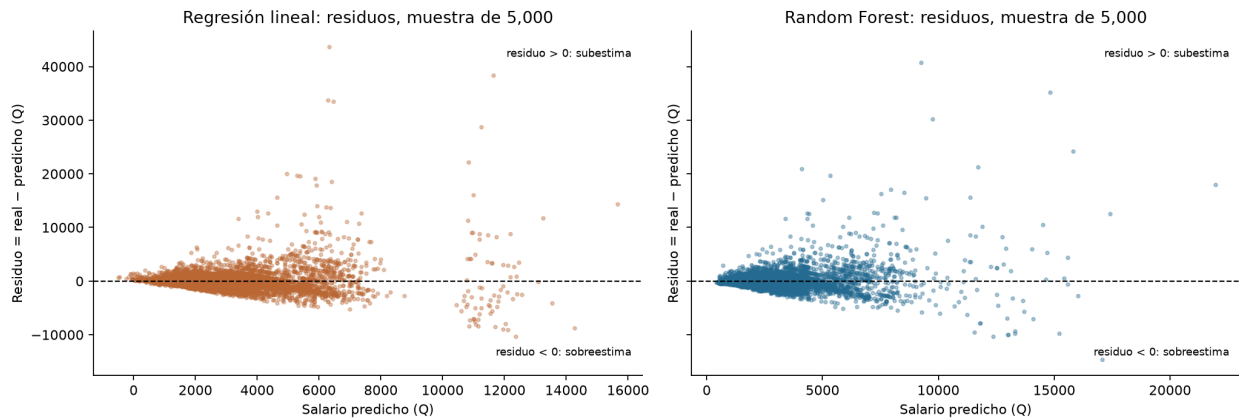


Figura 12. Residuos frente al salario predicho, con la línea horizontal en cero (misma muestra).

Tabla 13. Resumen de los residuos en todos los registros de 2026T1.

Modelo	Error medio	Residuo mediano	P5	P95	Asimetría	% subestimados
Regresión lineal	Q121	−Q31	−Q2,303	Q2,934	4.71	48.9%
Random Forest	Q110	−Q50	−Q2,076	Q2,618	4.31	47.8%

Los errores **no son simétricos**. El residuo mediano es negativo (−Q50 en Random Forest) pero el error medio es positivo (Q110), y la asimetría es 4.3: hay muchas sobreestimaciones moderadas y unas pocas subestimaciones muy grandes.

8.1 Errores por nivel educativo y dominio

Tabla 14. MAE y error medio por nivel educativo (todos los registros de 2026T1).

Nivel educativo	n	Salario mediano	MAE lineal	Error medio lineal	MAE RF	Error medio RF
Ninguno	1,016	Q1,680	Q824	Q109	Q628	Q50
Preprimaria	80	Q2,400	Q735	Q89	Q574	−Q21
Primaria	3,957	Q2,400	Q870	Q55	Q746	Q53
Básico	2,164	Q3,000	Q894	Q128	Q799	Q132
Diversificado	4,117	Q3,800	Q1,109	Q104	Q1,005	Q104
Superior	1,729	Q5,000	Q2,566	Q296	Q2,253	Q258
Maestría	183	Q10,000	Q5,553	−Q132	Q4,572	−Q213
Doctorado	12	Q18,500	Q12,919	Q6,063	Q10,606	Q6,442

Tabla 15. MAE y error medio por dominio (todos los registros de 2026T1).

Dominio	n	Salario mediano	MAE lineal	Error medio lineal	MAE RF	Error medio RF
Urbano Metropolitano	5,632	Q3,900	Q1,446	Q136	Q1,299	Q155
Resto Urbano	4,846	Q3,000	Q1,190	Q134	Q1,022	Q95
Rural Nacional	2,780	Q2,160	Q914	Q65	Q753	Q43

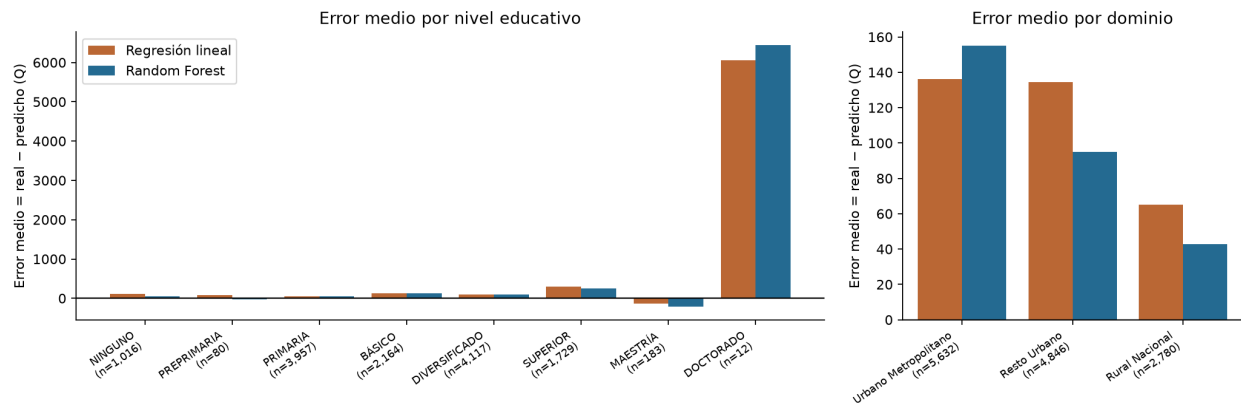


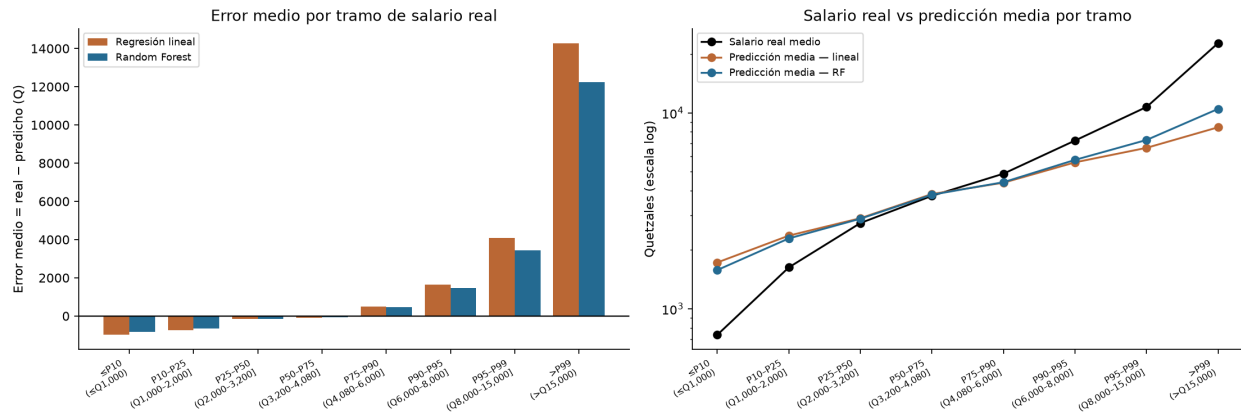
Figura 13. Error medio por nivel educativo y por dominio.

El error absoluto crece con el nivel educativo: el MAE de Random Forest va de Q628 para quienes no tienen estudios a Q2,253 con educación superior, y se dispara en maestría y doctorado. No es que el modelo "funcione peor" con esas personas: sus salarios son mucho más dispersos, y los grupos de posgrado son tan pequeños que deben leerse con cautela. Por dominio, el área metropolitana concentra el mayor MAE (Q1,299) y el área rural el menor (Q753).

8.2 Errores por percentil del salario

Tabla 16. Errores por tramo de percentil del salario real (todos los registros de 2026T1).

Tramo de salario real	n	Real medio	Pred. RF	Error medio lineal	Error medio RF	% subest. RF
≤P10 (≤Q1,000)	1,486	Q735	Q1,576	−Q987	−Q840	6%
P10–P25 (Q1,000–2,000]	2,546	Q1,626	Q2,286	−Q736	−Q660	23%
P25–P50 (Q2,000–3,200]	2,715	Q2,738	Q2,881	−Q161	−Q143	47%
P50–P75 (Q3,200–4,080]	3,200	Q3,770	Q3,821	−Q82	−Q51	56%
P75–P90 (Q4,080–6,000]	2,060	Q4,893	Q4,427	Q493	Q466	71%
P90–P95 (Q6,000–8,000]	608	Q7,225	Q5,756	Q1,634	Q1,469	84%
P95–P99 (Q8,000–15,000]	545	Q10,723	Q7,275	Q4,092	Q3,448	92%
>P99 (>Q15,000)	98	Q22,730	Q10,483	Q14,285	Q12,247	100%



Este análisis resumió todo lo anterior. En el 10% de salarios más bajos, los modelos **sobreestiman**: el salario real medio es Q735 y Random Forest predice en promedio Q1,576. Alrededor de la mediana el error medio es prácticamente cero. Pero por encima del percentil 99 el salario real medio es Q22,730 y la predicción media apenas Q10,483: **se subestima al 100% de esos registros**, con un error medio de Q12,247.

Gracias a esto pudimos entender el fenómeno de **regresión a la media**. Con sólo seis predictores, dos personas con la misma edad, antigüedad, horas, educación, categoría y dominio pueden ganar Q2,000 o Q20,000, y el modelo, que minimiza el error cuadrático, les asigna un valor intermedio. Como la distribución salarial es muy asimétrica, esa compresión produce muchos errores pequeños en los salarios bajos y pocos errores enormes en la cola alta, que son los que más pesan en el RMSE. Random Forest reduce el problema, pero no lo elimina.

9. Discusión y conclusiones

- **La preparación de datos definió todo lo demás.** Descubrir el archivo de 302 columnas, los 175 registros con un código de trimestre distinto y la diferencia entre "no corresponde" y "no respondió" evitó errores silenciosos que habrían contaminado cada análisis posterior.
- **El salario es muy asimétrico** (asimetría de 6.0) y conservamos sus extremos por instrucción. Esa forma de la distribución explica por qué el RMSE es mucho mayor que el MAE en todos los modelos.
- **Las variables numéricas se asocian débilmente con el salario**, pero la educación, la categoría ocupacional y el dominio marcan diferencias enormes. Los modelos lo confirmaron: la parte categórica cargó buena parte del poder predictivo.
- **La segmentación mostró cinco perfiles laborales** formados sin salario, con medianas salariales que difieren hasta 3.2 veces; la jornada parcial concentra los salarios más bajos.
- **Random Forest fue el mejor modelo** en validación y en la prueba final (R^2 de 0.547 en 2026), gracias a su capacidad de capturar interacciones. La regularización no ayudó a la regresión lineal porque no había sobreajuste que corregir.

- **Los modelos se sostuvieron en un período nuevo:** el error en 2026T1 fue incluso ligeramente menor que en la validación, lo que valida la división temporal que elegimos.
- **Los errores tienen una dirección clara:** se sobreestiman los salarios bajos y se subestiman sistemáticamente los altos, sobre todo por encima del percentil 95.

9.1 Limitaciones y posibles mejoras

Un R^2 cercano a 0.5 indica que la mitad de la variación salarial depende de factores que no estaban entre los seis predictores permitidos: la rama de actividad, el tamaño de la empresa, la ocupación específica o la experiencia previa. Además, los resultados no están ponderados por el factor de expansión, que sería necesario para producir estimaciones poblacionales; los salarios son nominales y las asociaciones que encontramos no son causales. Fuera de la comparación obligatoria, valdría la pena modelar el logaritmo del salario o su mediana mediante regresión cuantílica, e incorporar variables de ocupación y sector.

9.2 Lo que aprendimos de Spark

Más allá de los resultados, el laboratorio cambió nuestra forma de trabajar con datos. Aprendimos a pensar en transformaciones perezosas y acciones, a usar `cache` cuando un `DataFrame` se reutiliza muchas veces, a reconocer los *shuffles* de cada `groupBy` o `join` como los pasos más costosos, a guardar en Parquet desde el inicio y a calcular todo en Spark para llevar a pandas sólo lo necesario para dibujar. También entendimos que los pipelines de MLlib no son sólo una comodidad: son la forma más segura de garantizar que el preprocesamiento se aprenda únicamente con los datos de entrenamiento.

Anexo: reproducibilidad

El proyecto se ejecuta con Docker desde la carpeta `L07_Spark_ML_Lib`. Los datos se descargan del sitio del INE con el script incluido y los notebooks deben ejecutarse en orden, porque cada uno lee los archivos que genera el anterior:

```
python Data/descargar.py
docker compose up --build
```

<code>01_preparacion_eneic.ipynb</code>	Actividades 1-3
<code>02_segmentacion_kmeans.ipynb</code>	Actividad 4
<code>03_pipelines.ipynb</code>	Actividades 5-6
<code>04_evaluacion_final.ipynb</code>	Actividades 7-8

Fuente de datos: Instituto Nacional de Estadística de Guatemala, Encuesta Nacional de Empleo e Ingresos Continua (bases y diccionarios de Personas, 2025T1–2026T1).